1. The Blocking Problem: Diagnosis 2
1.1 Symptom Description . . . 2
1.2 Root Cause Analysis . . . 2
2. The SKILL.md Paradigm: Declarative Agent Boundaries 3
2.1 Origin: OpenClaw Standard . . . 4
2.2 SKILL.md Schema Specification . . . 4
2.3 How SKILL.md Solves the Blocking Problem . . . 5
3. The Tiered Policy Engine 6
3.1 Three-Tier Enforcement Model . . . 6
3.2 Policy Engine Implementation in Rust . . . 7
4. IPC Protocol Redesign 8
4.1 Transport Layer: gRPC over Unix Domain Sockets . . . 8
4.2 Structured Error Responses . . . 8
5. The Orchestrator AI: Kimi K2.7 Code Integration 9
5.1 Why Kimi K2.7 Code . . . 9
5.2 Orchestrator Architecture . . . 10
6. Full-Stack SEO Swarm: Agent Inventory 10
6.1 The 12-Agent Architecture . . . 10
6.2 Cross-Agent Communication Protocol . . . 11
7. Risk Analysis and Mitigation 12
7.1 Residual Risks After Redesign . . . 12
8. Implementation Roadmap 13
8.1 Phase 1: Foundation (Weeks 1-3) . . . 13
8.2 Phase 2: Tiered Enforcement (Weeks 4-6) . . . 13
8.3 Phase 3: Orchestrator Integration (Weeks 7-9) . . . 14
8.4 Phase 4: Hardening and Production (Weeks 10-12) . . . 14
9. Deliverable File Map 15

1. The Blocking Problem: Diagnosis

1.1 Symptom Description

The Eli-OS Rust control plane was designed as a fail-closed safety kernel that governs all interactions between the human operator, the Orchestrator AI, and the Python-based Eli Claw agent swarm. In practice, the control plane has become the single largest bottleneck in the system. When a human operator issues a command, or when the Orchestrator AI routes a task to a specialized agent, the Rust kernel intercepts the Inter-Process Communication (IPC) request and evaluates it against a set of hardcoded policy rules. If any rule fails, the kernel returns a **PermissionDenied** error and halts the operation entirely. This design was intentionally conservative to prevent rogue agent behavior, but it has created a regime where legitimate, well-intentioned operations are routinely blocked.

The blocking manifests in three distinct failure modes that have been observed during development and pilot testing. First, **over-scoped policy checks** occur when the kernel evaluates an IPC request against policy rules that were designed for a different agent or a different operational context. For example, when the Technical SEO agent requests access to the crawl_results table, the kernel may block the request because a separate policy rule intended for the Parasite SEO agent restricts write access to that same table. The kernel does not distinguish between agents; it applies a monolithic policy to all IPC traffic, resulting in false-positive denials that halt productive work.

Second, **cascading blocks** occur when a single blocked operation causes downstream failures across the swarm. When the Orchestrator AI decomposes a complex task and routes sub-tasks to multiple agents, a block on one agent can cause the entire workflow to stall. The Orchestrator has no mechanism to retry, reroute, or de-escalate a blocked operation. It simply receives a PermissionDenied error and reports failure to the human operator, who must then manually diagnose which policy rule caused the block and either override it or restructure the task. This manual intervention cycle defeats the purpose of an autonomous multi-agent system.

Third, **opaque error reporting** makes diagnosis difficult. When the kernel blocks an IPC request, it returns a generic PermissionDenied error without specifying which policy rule was violated, why the rule exists, or what the operator can do to resolve the conflict. This lack of transparency turns every block into a debugging session that requires the operator to inspect the Rust kernel source code, understand the policy engine internals, and manually trace the IPC request path. The result is a control plane that inspires frustration rather than confidence, and that actively undermines the velocity of the development and operations team.

1.2 Root Cause Analysis

The root cause of the blocking problem is architectural: the Rust control plane uses a **monolithic, hardcoded policy engine** that was not designed for a multi-agent swarm. The original design assumed a single Python application (Eli Claw) communicating with a single Rust kernel. Policy rules were written as

Rust match statements and if-else chains, embedded directly in the kernel binary. This approach provided strong safety guarantees for a single-agent system, but it does not scale to a swarm of twelve or more specialized agents, each with distinct capabilities, knowledge bases, and tool access requirements.

The fundamental architectural flaw is the **separation of policy from capability**. In the current system, the Rust kernel owns both the enforcement mechanism (the IPC interceptor) and the policy definition (the hardcoded rules). The Python agents have no way to declare their own capabilities or boundaries. They cannot tell the kernel what they are allowed to do; they can only discover what they are not allowed to do by attempting an operation and receiving a PermissionDenied error. This inverted model places the entire burden of policy management on the kernel, making it a single point of failure and a single point of bottleneck for the entire system.

Furthermore, the current IPC mechanism lacks a **tiered enforcement model**. Every IPC request, regardless of its risk level, is subjected to the same exhaustive policy evaluation. A low-risk read operation (such as an agent querying its own knowledge base) receives the same level of scrutiny as a high-risk write operation (such as an agent modifying shared state in the PostgreSQL database). This flat enforcement model introduces unnecessary latency for the vast majority of operations while providing no additional safety benefit, because the high-risk operations that actually warrant strict scrutiny are processed through the same fast path as everything else.

Table 1: Blocking Failure Modes

Failure Mode	Description	Impact	Current Mitigation
Over-scoped Policy	Monolithic rules applied across all agents regardless of context	False-positive denials on legitimate operations	None - manual kernel code change required
Cascading Blocks	Single block stalls entire multi-agent workflow	Workflow failure requiring full restart	None - Orchestrator cannot retry or reroute
Opaque Errors	PermissionDenied with no policy rule reference	Debugging requires source code inspection	None - no error detail in IPC response
Flat Enforcement	All IPC requests undergo same scrutiny level	Unnecessary latency on low-risk reads	None - no risk-tier classification exists
Static Policy	Rules compiled into Rust binary at build time	Policy changes require kernel recompilation	None - no runtime policy update mechanism

2. The SKILL.md Paradigm: Declarative Agent Boundaries

2.1 Origin: OpenClaw Standard

The solution to the blocking problem draws from the OpenClaw project, an open-source multi-channel gateway for AI agents. OpenClaw introduced the **SKILL.md** standard, a structured Markdown file that teaches an AI agent how to use a tool or perform a function. Each SKILL.md file serves as a declarative specification of an agent's identity, capabilities, knowledge base scope, forbidden actions, input/output schemas, system prompt invariants, and IPC policy. The critical innovation is that the agent's behavioral boundaries are defined in a machine-readable document that both the agent and the control plane can parse and enforce.

By adopting the SKILL.md paradigm, Eli-OS shifts from a model where the Rust kernel *imposes* boundaries on Python agents to a model where agents *declare* their own boundaries, and the kernel *verifies* compliance. This inversion of responsibility has profound implications for system architecture. The kernel no longer needs to contain hardcoded knowledge of what each agent is allowed to do. Instead, it reads the SKILL.md files at startup, builds a dynamic capability manifest, and uses that manifest as the basis for all policy enforcement decisions. When an agent needs to change its capabilities (for example, adding a new tool or accessing a new database table), the operator updates the SKILL.md file and signals the kernel to reload, without recompiling any Rust code.

2.2 SKILL.md Schema Specification

Each SKILL.md file in the Eli-Swarm-Claw project follows a standardized schema with eight mandatory sections. The **Identity** section declares the agent's name, role, domain, and version, providing the kernel with a unique identifier for policy lookups. The **Purpose** section provides a natural-language description of the agent's function, used by the Orchestrator AI for task routing decisions. The **Knowledge Base Scope** section defines the bounded corpus of information the agent is authorized to draw from, including specific data sources, explicit exclusions, and a refresh policy that determines how stale data is tolerated.

The **Capabilities (Tools)** section lists every tool, function, or API endpoint the agent is authorized to invoke, with a brief description of each. The **Forbidden Actions** section explicitly enumerates actions the agent must never perform, including cross-domain table access and cross-agent endpoint invocation. The **Input Schema** and **Output Schema** sections define the expected formats for agent communication. The **Constraints** section sets the system prompt invariant (which forces the agent to answer only from its authorized knowledge base), maximum output tokens, and temperature. Finally, the **IPC Policy** section specifies the exact PostgreSQL tables the agent can read from or write to, the API endpoints it can call, its resource limits (memory, CPU, maximum duration), and the conditions under which it must escalate to the Orchestrator or human operator.

Table 2: SKILL.md Schema Sections

Section	Purpose	Enforced By	Modifiable At Runtime
Identity	Unique agent identifier, role, domain, version	Rust kernel (manifest key)	No (requires reload)
Knowledge Base Scope	Authorized data sources, exclusions, refresh policy	Rust kernel (RAG guard)	Yes (hot reload)
Capabilities (Tools)	List of authorized tools with descriptions	Rust kernel (tool whitelist)	Yes (hot reload)
Forbidden Actions	Explicit action prohibitions	Rust kernel (deny list)	Yes (hot reload)
Input/Output Schema	Communication format definitions	Python agent (validation)	Yes (hot reload)
Constraints	System prompt invariant, tokens, temperature	Python agent (SLM config)	Yes (hot reload)
IPC Policy	Table access, endpoints, resource limits	Rust kernel (policy engine)	Yes (hot reload)
Escalation Triggers	Conditions requiring Orchestrator/human intervention	Rust kernel (escalation)	Yes (hot reload)

2.3 How SKILL.md Solves the Blocking Problem

The SKILL.md paradigm directly addresses all five failure modes identified in the diagnosis. **Over-scoped policies** are eliminated because each agent's policy is defined in its own SKILL.md file, scoped exclusively to its domain. The Technical SEO agent's SKILL.md declares access to technical_seo_audits and crawl_results tables; the Parasite SEO agent's SKILL.md declares access to parasite_analyses and backlink_profiles. The kernel no longer applies a monolithic policy; it evaluates each IPC request against the requesting agent's specific SKILL.md manifest. A request from the Technical SEO agent to write to the backlink_profiles table is immediately denied not because of a global rule, but because the Technical SEO agent's own SKILL.md does not list that table in its IPC Policy.

Cascading blocks are mitigated because the SKILL.md schema includes Escalation Triggers, which define the conditions under which an agent should escalate to the Orchestrator rather than failing silently. When an agent encounters an operation it cannot perform, it raises an escalation event with structured context (what it tried to do, why it failed, what it needs). The Orchestrator can then make an intelligent decision: retry with modified parameters, reroute to a different agent, compose a multi-agent response, or escalate to the human operator with a clear diagnosis. This replaces the current binary block/pass model with a gradient of responses that keeps workflows moving.

Opaque errors are replaced by structured IPC responses that include the specific SKILL.md section that was violated, the agent's declared capability, and the requested action. When the kernel denies an IPC

request, it returns a PolicyViolation response (not a generic PermissionDenied) that contains: the agent identity, the violated policy section, the exact rule text from the SKILL.md file, and a suggested resolution. This transforms debugging from a source-code archaeology exercise into a straightforward policy adjustment.

3. The Tiered Policy Engine

3.1 Three-Tier Enforcement Model

The redesigned Eli-OS control plane replaces the flat enforcement model with a three-tier system that classifies every IPC request by risk level and applies proportionate scrutiny. This ensures that low-risk operations (the vast majority of agent traffic) are processed with minimal latency, while high-risk operations receive the thorough evaluation they warrant. The three tiers are: **Green (autonomous pass)**, **Amber (logged and sampled)**, and **Red (mandatory human approval)**. Each tier has distinct processing rules, latency characteristics, and audit requirements.

The **Green tier** handles read-only operations within an agent's declared domain. When the Keyword Agent queries the keyword_clusters table to retrieve previously computed clusters, the kernel verifies that keyword_clusters is listed in the Keyword Agent's SKILL.md IPC Policy under allowed read tables, confirms the operation is read-only, and passes the request through without further evaluation. Green tier operations complete in under one millisecond of added latency, which is negligible compared to the PostgreSQL query time. The kernel logs the operation for audit purposes but does not generate an alert. Green tier operations account for an estimated 80-85% of all IPC traffic in a typical SEO workflow.

The **Amber tier** handles write operations and cross-agent data access. When the Technical SEO Agent writes a new audit result to the tech_seo_audits table, the kernel verifies the write permission, checks resource limits (is the agent within its memory and CPU allocation?), validates the data schema against the SKILL.md output schema, and logs the operation with full context. The kernel also applies statistical sampling: a configurable percentage (default 10%) of Amber operations are flagged for asynchronous review by the QA Agent. This provides continuous quality assurance without blocking the workflow. Amber tier operations add approximately 5-10 milliseconds of latency, which is acceptable for write operations that already incur PostgreSQL write latency.

The **Red tier** handles operations that are irreversible, high-impact, or cross-domain. Examples include: deleting data from shared tables, modifying another agent's records, executing external API calls that have cost implications (such as paid SERP API queries), or any operation that exceeds the agent's declared resource limits. Red tier operations are not automatically blocked; instead, they are routed to the human operator's approval queue with full context: the agent identity, the requested operation, the SKILL.md policy section, the risk classification rationale, and a recommended action (approve, deny, or modify). The human operator can approve the operation with a single click, deny it with a reason that feeds back into the agent's

learning loop, or modify the parameters and return it to the agent for re-execution.

Table 3: Tiered Enforcement Specification

Tier	Trigger Conditions	Processing	Latency	Audit
Green	Read-only, within agent domain, within resource limits	Verify SKILL.md read permission, pass	<1ms added	Log only, no alert
Amber	Write operations, cross-agent reads, resource threshold approaching	Verify write permission, check limits, validate schema, sample for QA	5-10ms added	Full context log, 10% async QA review
Red	Irreversible ops, cross-domain writes, external paid APIs, resource limit exceeded	Halt, route to human approval queue with full context	Blocking (human-in-loop)	Full audit trail, escalation record, approval receipt

3.2 Policy Engine Implementation in Rust

The policy engine is implemented as a dedicated Rust crate within the eli-os workspace: **eli-policy**. This crate is responsible for parsing SKILL.md files into a structured CapabilityManifest data structure, evaluating IPC requests against the manifest, classifying requests into enforcement tiers, and generating structured PolicyViolation responses when a request is denied. The manifest is loaded at kernel startup and can be hot-reloaded at runtime by sending a SIGUSR1 signal to the kernel process or by calling the /api/v1/kernel/reload-manifest endpoint.

The core data structure, CapabilityManifest, is a Serde-serializable struct that maps each agent identity to its parsed SKILL.md sections. The SkillParser module reads each SKILL.md file from a configured directory (default: /etc/eli-os/skills/), extracts the structured sections using a combination of regex-based line parsers and a lightweight Markdown AST, and constructs the CapabilityManifest. If any SKILL.md file fails to parse (missing mandatory sections, malformed IPC Policy, etc.), the parser logs a structured error and excludes that agent from the manifest, rather than failing the entire kernel startup. This ensures that a syntax error in one agent's SKILL.md does not prevent the rest of the swarm from operating.

The PolicyEvaluator module is the heart of the tiered enforcement system. It receives an IpcRequest (containing the agent identity, the requested operation type, the target resource, and the payload), looks up the agent's CapabilityManifest entry, and executes a three-step evaluation. First, it checks the operation against the agent's Forbidden Actions list; if the operation matches any forbidden pattern, it returns an immediate Red-tier denial. Second, it checks the operation against the IPC Policy (allowed tables, endpoints, resource limits); if the operation is outside the declared policy, it returns an Amber-tier denial with the specific policy section reference. Third, it classifies the operation into Green, Amber, or Red based on the tier rules described above. The entire evaluation path is deterministic, has O(1) complexity for manifest lookups, and O(n) complexity for forbidden action matching (where n is the number of forbidden actions per

agent, typically fewer than ten).

4. IPC Protocol Redesign

4.1 Transport Layer: gRPC over Unix Domain Sockets

The current Eli-OS prototype uses a basic HTTP-based IPC mechanism between the Rust kernel and the Python Eli Claw application. This approach has two critical limitations: serialization overhead (JSON encoding/decoding adds 2-5ms per request) and lack of strong typing (JSON does not enforce schema compliance at the transport level). The redesigned IPC protocol uses **gRPC over Unix Domain Sockets** (UDS) as the transport layer. gRPC provides Protocol Buffer-based serialization, which is 3-10x faster than JSON for structured data and enforces schema compliance at compile time. Unix Domain Sockets provide inter-process communication on the same machine with lower latency than TCP loopback (no network stack traversal) and built-in file-system-level access control.

The gRPC service definition (in protobuf format) specifies four core RPC methods. The **EvaluateRequest** method accepts an IpcRequest message and returns an IpcResponse message containing either an approval (with optional rate-limit metadata) or a PolicyViolation with structured error details. The **ReportResult** method allows agents to report operation results back to the kernel for audit logging and event broadcasting. The **Escalate** method allows agents to raise escalation events when they encounter conditions defined in their SKILL.md Escalation Triggers. The **Heartbeat** method provides a lightweight keep-alive mechanism that the kernel uses to monitor agent health and detect stalled operations.

The choice of gRPC over alternative IPC mechanisms was driven by several factors specific to the Eli-Swarm-Claw architecture. First, gRPC's streaming support enables the kernel to push policy updates to agents in real time without requiring polling. When a SKILL.md file is updated and the manifest is reloaded, the kernel can stream the updated capability definition to the affected agent, which then adjusts its behavior without restarting. Second, gRPC's built-in deadline and cancellation semantics allow the kernel to enforce the max_duration_seconds resource limit from the SKILL.md IPC Policy. When an agent's operation exceeds its time allocation, the kernel cancels the gRPC stream and returns a ResourceExceeded response. Third, Protocol Buffer's backward compatibility guarantees ensure that the IPC protocol can evolve independently of the kernel and agent codebases, supporting incremental rollout of new features without breaking existing deployments.

4.2 Structured Error Responses

The redesigned IPC protocol replaces the generic PermissionDenied error with a structured **PolicyViolation** response that provides full diagnostic context. Every denial includes: the agent identity that made the request, the operation type (read, write, execute, delete), the target resource (table name, endpoint URL, or tool

identifier), the violated SKILL.md section name (e.g., "IPC Policy: Allowed Tables"), the exact rule text from the SKILL.md file that was violated, the enforcement tier that evaluated the request (Green, Amber, or Red), a human-readable explanation of why the violation occurred, and a suggested resolution (update SKILL.md, escalate to Orchestrator, request human override, or modify the operation parameters). This structured response format transforms every policy denial from a debugging obstacle into an actionable diagnostic.

Table 4: IPC Mechanism Comparison

Mechanism	Serialization	Latency (Local)	Schema Enforcement	Streaming	Access Control
HTTP/JSON (current)	JSON	2-5ms overhead	Runtime only	No	Application-level
gRPC/UDS (proposed)	Protocol Buffers	0.2-0.5ms overhead	Compile-time + runtime	Bidirectional	File-system + protocol
Shared Memory	Raw bytes	<0.1ms overhead	None (manual)	No	OS process isolation
TCP Sockets	Protocol Buffers	0.5-1ms overhead	Compile-time + runtime	Bidirectional	Network-level

5. The Orchestrator AI: Kimi K2.7 Code Integration

5.1 Why Kimi K2.7 Code

The Orchestrator AI is the "second AI" in the Eli-OS architecture, responsible for decomposing complex full-stack SEO tasks, routing sub-tasks to the appropriate specialized agents, synthesizing multi-agent outputs into unified responses, and handling escalation events from the swarm. The selection of **Kimi K2.7 Code** as the Orchestrator model is driven by four key technical capabilities that align precisely with the Eli-OS requirements.

First, Kimi K2.7 Code is an **open-weight model** released under permissive licensing by Moonshot AI. This means the Orchestrator can be hosted entirely within VirtuaLab Digital's sovereign infrastructure, without sending sensitive SEO data, client URLs, or competitive intelligence to closed-source API providers. The model weights are fully auditable, allowing the team to inspect and modify the model's behavior if needed. Second, Kimi K2.7 Code is explicitly designed as a **terminal-first agentic model**, optimized for executing multi-step workflows, managing tool calls, and handling complex decision trees. This makes it ideally suited

for the Orchestrator role, which requires precisely these capabilities: receiving a high-level task, breaking it into sub-tasks, executing tool calls to delegate to agents, and handling the results.

Third, Kimi K2.7 Code supports a **1-million-token context window** (via the Kimi-Linear architecture), which is critical for the Orchestrator's role. A full-stack SEO audit of a large website can generate hundreds of pages of crawl data, keyword research, entity analysis, and competitor intelligence. The Orchestrator must be able to hold this context in memory while making routing and synthesis decisions. With a 1M token context, the Orchestrator can ingest the complete output of multiple agents simultaneously without truncation or summarization loss. Fourth, the model is the first open-weight model available in **GitHub Copilot's model picker**, demonstrating production readiness and broad community validation.

5.2 Orchestrator Architecture

The Orchestrator operates as a stateful decision engine within the Eli Claw Python application plane. It is not a separate process; it is a Python class (EliOrchestrator) that wraps the Kimi K2.7 Code model (served locally via vLLM or SGLang for high-throughput inference) and provides four core methods: **decompose**, **route**, **synthesize**, and **handle_escalation**. The decompose method accepts a high-level task from the human operator or the API, uses the Kimi model to break it into a directed acyclic graph (DAG) of sub-tasks, and returns the DAG with dependency edges. The route method accepts a sub-task, evaluates it against each agent's SKILL.md Purpose section, and selects the most capable agent. The synthesize method accepts the outputs of completed sub-tasks and merges them into a unified response. The handle_escalation method accepts an escalation event from an agent or the Rust kernel and decides whether to retry, reroute, compose, or escalate to the human operator.

The Orchestrator does not bypass the Rust control plane. Every sub-task delegation and agent invocation passes through the kernel's IPC evaluation. The Orchestrator's advantage is intelligence, not authority. It can make routing decisions that the kernel cannot (because routing requires semantic understanding of task intent, which is beyond the kernel's scope), but it cannot override the kernel's policy enforcement. This separation of concerns is critical: the Orchestrator provides **semantic safety** (ensuring tasks are routed to the right agent and outputs are coherent), while the kernel provides **hard safety** (ensuring agents stay within their declared boundaries). Neither layer is sufficient alone; together, they provide comprehensive governance without the blocking problem.

6. Full-Stack SEO Swarm: Agent Inventory

6.1 The 12-Agent Architecture

The Eli Claw application plane hosts twelve specialized agents, each defined by its own SKILL.md file and governed by the tiered policy engine. The swarm is designed so that the full-stack SEO capability emerges

from the composition of these specialized agents, orchestrated by the Kimi K2.7 Code Orchestrator. No single agent is "gigantic"; each is a constrained Micro-RAG system with a bounded knowledge base, a specific tool set, and clear escalation paths. The "full-stack" capability is achieved through coordination, not through a single monolithic model.

Table 5: Agent Inventory and Domain Mapping

Agent	Domain	Knowledge Base	Primary Tools	Tier Focus
Technical SEO	Crawlability, HTTP, CWV	Google Search Central, W3C, Core Web Vitals specs	Sitemap parser, HTTP inspector, robots.txt analyzer	Green reads, Amber writes
On-Page SEO	Meta tags, schema, content	Schema.org, NLP keyword semantics	DOM parser, meta-tag generator, readability scorer	Green reads, Amber writes
Parasite SEO	High-DA platforms, backlinks	Platform TOS, backlink velocity data	DA checker, platform API wrappers, SERP tracker	Amber reads, Red writes
GEO	AI citations, entity salience	SGE/Perplexity patterns, citation data	Citation tracker, entity-salience analyzer	Green reads, Amber writes
AI Citation	Brand mentions in AI	AI answer logs, citation source data	Citation probe, mention-frequency analyzer	Green reads, Green writes
Keyword	Seed expansion, clustering	Search demand data, SERP features	Keyword expander, intent classifier, clusterer	Green reads, Amber writes
Entity	Entity extraction, semantics	Knowledge graph data, semantic web	Entity extractor, relationship mapper	Green reads, Amber writes
Competitor	Keyword/content/visibility gaps	Competitor crawl data, SERP snapshots	Gap analyzer, visibility benchmark	Green reads, Amber writes
Local SEO	Service areas, NAP, GBP	GBP signals, local SERP data	NAP checker, service-area mapper	Green reads, Amber writes
Indexing	Sitemaps, IndexNow, feeds	GSC coverage data, feed specs	Sitemap generator, IndexNow pusher	Amber reads, Red writes
QA	Data quality, compliance	Policy rules, validation schemas	Data validator, output checker	Green reads (all tables, read-only)
Report	Client-facing summaries	All agent outputs (read-only)	Report generator, chart builder	Green reads, Amber writes

6.2 Cross-Agent Communication Protocol

Agents do not communicate directly with each other. All inter-agent communication is mediated by the Rust kernel through an **event bus** architecture. When an agent completes a task, it publishes a result event to the kernel's event bus. The kernel logs the event, evaluates it against the QA Agent's sampling policy, and broadcasts it to any subscribers. The Orchestrator subscribes to all agent result events and uses them to update its task DAG. Other agents may subscribe to specific event types (for example, the Entity Agent subscribes to Keyword Agent results to enrich its knowledge graph), but they never receive direct messages from other agents.

This event bus architecture prevents several failure modes that are common in direct agent-to-agent communication systems. First, it eliminates **message amplification loops**, where Agent A sends a message to Agent B, which triggers a response to Agent A, which triggers another response, creating an unbounded cycle. The kernel's event bus is a one-directional publish-subscribe system; agents can publish results and subscribe to events, but they cannot reply to events. If an agent needs to act on another agent's output, it must create a new task (which goes through the kernel's IPC evaluation) rather than sending a direct response. Second, it provides a **single audit trail** for all inter-agent communication. Every event is logged with a ULID, a timestamp, the publisher identity, the subscriber identities, and the payload. This audit trail is critical for debugging, compliance, and the QA Agent's asynchronous review process.

7. Risk Analysis and Mitigation

7.1 Residual Risks After Redesign

While the SKILL.md paradigm and tiered policy engine eliminate the five identified failure modes, they introduce new risks that must be acknowledged and mitigated. The most significant residual risk is **SKILL.md drift**: the possibility that an agent's SKILL.md file becomes outdated relative to its actual behavior. If an agent is updated to use a new database table but its SKILL.md is not updated accordingly, the kernel will deny the legitimate IPC request, reproducing the original blocking problem through a different mechanism. Mitigation: the CI/CD pipeline must include a SKILL.md validation step that compares the agent's actual database queries (extracted from integration tests) against its declared IPC Policy. Any discrepancy blocks the deployment.

A second residual risk is **Orchestrator hallucination in routing**. The Kimi K2.7 Code model, like all large language models, can produce incorrect outputs. If the Orchestrator routes a technical SEO task to the Parasite SEO agent, the Parasite agent will either refuse (because the task is outside its declared Purpose) or produce a low-quality result. The kernel does not validate the Orchestrator's routing decisions because routing is a semantic operation that requires understanding task intent. Mitigation: the QA Agent asynchronously reviews a sample of routing decisions and flags anomalies. Additionally, each agent's system prompt invariant forces it to refuse tasks outside its domain, providing a secondary safety net.

A third residual risk is **gRPC serialization bottleneck** under high concurrency. While gRPC is significantly faster than JSON for individual requests, the Python gRPC client has a higher per-connection overhead than the current HTTP client. If the swarm scales to hundreds of concurrent agents, the gRPC connection pool may become a bottleneck. Mitigation: the IPC architecture supports connection multiplexing through gRPC's HTTP/2 transport, and the kernel can shard the IPC load across multiple Unix Domain Socket listeners if needed. Performance benchmarks should be conducted at the target scale before production deployment.

Table 6: Residual Risk Register

Risk	Probability	Impact	Mitigation Strategy	Owner
SKILL.md drift	Medium	High	CI/CD validation step comparing test queries to IPC Policy	DevOps
Orchestrator routing hallucination	Medium	Medium	QA Agent sampling + agent domain refusal invariant	ML Engineering
gRPC bottleneck at scale	Low	High	Connection multiplexing + UDS sharding + benchmarks	Platform Engineering
Hot-reload race condition	Low	Medium	Atomic manifest swap with read-copy-update pattern	Rust Kernel Team
Kimi K2.7 inference latency	Medium	Medium	vLLM/SGLang batching + model quantization (INT4)	ML Engineering

8. Implementation Roadmap

8.1 Phase 1: Foundation (Weeks 1-3)

The first phase establishes the foundational components that all subsequent work depends on. The primary deliverables are: the SKILL.md schema parser in Rust (eli-skill-parser crate), the CapabilityManifest data structure with Serde serialization, and a minimal policy evaluator that supports Green-tier enforcement only. This phase also includes writing the SKILL.md files for three pilot agents (Technical SEO, Keyword, and QA) and integrating them into the existing Eli Claw FastAPI scaffold. The acceptance criterion for Phase 1 is a successful end-to-end test where a Python agent makes a gRPC IPC request to the Rust kernel, the kernel evaluates it against the agent's SKILL.md manifest, and returns an approval or a structured PolicyViolation response.

8.2 Phase 2: Tiered Enforcement (Weeks 4-6)

The second phase extends the policy evaluator to support all three enforcement tiers (Green, Amber, Red) and implements the human approval queue for Red-tier operations. This phase also includes the structured error response format, the event bus architecture for cross-agent communication, and the SIGUSR1 hot-reload mechanism for SKILL.md files. The remaining nine SKILL.md files are written and integrated. The acceptance criterion is a multi-agent workflow where the Orchestrator decomposes a task, routes sub-tasks to three different agents, one of which triggers an Amber-tier log and another triggers a Red-tier human approval, and the workflow completes successfully with full audit trail.

8.3 Phase 3: Orchestrator Integration (Weeks 7-9)

The third phase integrates the Kimi K2.7 Code model as the Orchestrator AI. This phase includes setting up the vLLM inference server, implementing the EliOrchestrator Python class with its four core methods (decompose, route, synthesize, handle_escalation), connecting the Orchestrator to the kernel's event bus, and implementing the escalation handling flow. The acceptance criterion is a full-stack SEO workflow triggered by a natural-language prompt (for example, "Audit example.com for technical SEO issues, research keywords, and generate a content brief"), which the Orchestrator decomposes, routes to the appropriate agents, and synthesizes into a unified report without human intervention.

8.4 Phase 4: Hardening and Production (Weeks 10-12)

The final phase focuses on production readiness: performance benchmarking of the gRPC IPC layer under load, security audit of the policy engine and SKILL.md parser, comprehensive integration testing across all twelve agents, documentation of the SKILL.md authoring guide for future agent developers, and deployment of the complete system to the VirtuaLab Digital infrastructure with Docker Compose orchestration. The acceptance criterion is a 72-hour soak test where the system processes a continuous stream of SEO tasks across all twelve agents without a single false-positive block or policy engine failure.

Table 7: Implementation Timeline

Phase	Duration	Key Deliverables	Acceptance Criterion
1: Foundation	Weeks 1-3	SKILL.md parser, CapabilityManifest, 3 pilot agents, gRPC IPC	End-to-end Green-tier IPC test passes
2: Tiered Enforcement	Weeks 4-6	Amber/Red tiers, event bus, hot-reload, 12 SKILL.md files	Multi-agent workflow with all three tiers
3: Orchestrator	Weeks 7-9	Kimi K2.7 Code integration, EliOrchestrator, escalation flow	Natural-language full-stack SEO workflow
4: Production	Weeks 10-12	Benchmarks, security audit, docs, 72-hour soak test	Zero false-positive blocks in 72-hour test

9. Deliverable File Map

This white paper is accompanied by a set of implementation deliverables that provide the concrete code artifacts needed to execute the architectural redesign described in this document. The deliverables are organized into four directories: the SKILL.md templates (twelve files, one per agent), the Rust control plane code (three crates: skill parser, policy engine, and IPC handler), the Python integration code (agent base class, IPC client, and Orchestrator), and integration notes that map the OpenClaw and Kimi K2.7 Code ecosystems to the Eli-OS architecture.

Table 8: Deliverable File Structure

Path	Description	Language
skill-templates/technical_seo.md	Technical SEO Agent SKILL.md	Markdown
skill-templates/on_page_seo.md	On-Page SEO Agent SKILL.md	Markdown
skill-templates/parasite_seo.md	Parasite SEO Agent SKILL.md	Markdown
skill-templates/geo_agent.md	GEO Agent SKILL.md	Markdown
skill-templates/ai_citation.md	AI Citation Agent SKILL.md	Markdown
skill-templates/keyword_agent.md	Keyword Agent SKILL.md	Markdown
skill-templates/entity_agent.md	Entity Agent SKILL.md	Markdown
skill-templates/competitor_agent.md	Competitor Agent SKILL.md	Markdown
skill-templates/local_seo.md	Local SEO Agent SKILL.md	Markdown
skill-templates/indexing_agent.md	Indexing Agent SKILL.md	Markdown
skill-templates/qa_agent.md	QA Agent SKILL.md	Markdown
skill-templates/report_agent.md	Report Agent SKILL.md	Markdown
rust-control-plane/eli-skill-parser/	Rust SKILL.md parser crate	Rust
rust-control-plane/eli-policy-engine/	Rust tiered policy engine crate	Rust
rust-control-plane/eli-ipc-handler/	Rust gRPC IPC handler crate	Rust
python-integration/agents/	Python agent base class + examples	Python
python-integration/orchestrator/	Python EliOrchestrator implementation	Python
integration-notes/	OpenClaw + Kimi K2.7 mapping document	Markdown